

Dataset
- feat_data : pd.DataFrame - target_data : pd.Series - feature_names : List[str] - target_name : str
+ init(file_path: str) + compute_mutual_information() : Dict[str, float] + sort_features_by_mi(mi_dict: Dict[str, float]) : Dataset + train_test_split(test_size: float, random_state: int) : Tuple[Dataset, Dataset] + row_to_text(row_index: int, separator: str) : str + len() : int

feat_data — il DataFrame pandas che contiene le 2381 colonne feature. È il "corpo" del dataset, senza la colonna label.

target_data — la Series pandas con i valori 0/1.

feature_names — lista ordinata dei nomi simbolici. Mantiene la corrispondenza tra indice colonna e nome.

target_name — stringa con il nome dell'ultima colonna del csv (label). Viene ricavata automaticamente alla lettura.

__init__(file_path) — apre il csv, ricava target_name dall'ultima colonna, separa feature e target e li carica in memoria. file_path non viene conservato.

compute_mutual_information() — calcola quanto ogni feature è utile per distinguere malware da goodwill. Restituisce un dizionario ordinato dal più importante al meno.

sort_features_by_mi(mi_dict) — prende il dizionario prodotto dal metodo precedente e restituisce un nuovo Dataset con le colonne riordinate per importanza decrescente.

train_test_split(test_size, random_state) — divide i dati in training e test in modo stratificato, garantendo la stessa proporzione malware/goodware in entrambi. Restituisce due oggetti Dataset.

row_to_text(row_index, separator) — converte una riga in una stringa testuale del tipo "byte_histogram_0: 0.021, string_0: 3.000, ...".

__len__() —permette di scrivere di ottenere il numero di esempi.

RISULTATI DEI TEST

test_feature_count.py

Conferma che il numero di feature è 2381 e mostra le prime 10 (byte_histogram_0, byte_histogram_1, ...)

test_ml.py

Esegue il test sia sul file di training che sul file di test: caricamento, calcolo della MI, ordinamento delle feature, split stratificato e conversione in testo.